

Exp No.: 3 Implementing Asset Management with Hyperledger Fabric

Aim

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

Software Requirements

- ☐ Kali Linux / Ubuntu
- ☐ Docker
- ☐ Docker Compose
- ☐ Node.js (Version 18 or later)
- ☐ npm
- ☐ Git
- ☐ Hyperledger Fabric Samples
- ☐ Hyperledger Fabric Binaries
- ☐ Hyperledger Fabric Docker Images
- ☐ Visual Studio Code (Optional)

Hardware Requirements

- ☐ Processor: Intel Core i3 or above
- ☐ RAM: Minimum 8 GB
- ☐ Storage: Minimum 20 GB Free Disk Space
- ☐ Internet Connection

Procedure

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Expected Output:

Creating channel 'mychannel'

Starting peer0.org1

Starting peer0.org2

Starting orderer.example.com

Network Started Successfully

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```

Expected Output

Packaging Chaincode

Installing Chaincode

Approving Chaincode

Committing Chaincode

Chaincode committed successfully

Step 4: Initialize the Ledger

Populate the ledger with sample assets.

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $ORDERER_CA \  
-C mychannel \  
-n basic \  
-c '{"function":"InitLedger","Args":[]}'
```

Expected Output

Ledger Initialized Successfully

Step 5: Query All Assets

Retrieve all asset records stored in the blockchain.

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

Expected Output

```
[  
{  
  "ID":"asset1",  
  "Color":"blue",  
  "Owner":"Tomoko",  
  "Size":5,  
  "AppraisedValue":300  
},  
{
```

```
"ID": "asset2",
"Color": "red",

"Owner": "Brad",
"Size": 5,
"AppraisedValue": 400
}
]
```

Step 6: Create a New Asset

Create a new asset named asset11.

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls \

--cafile \$ORDERER_CA \

-C mychannel \

-n basic \

-c '{"function": "CreateAsset", "Args": ["asset11", "White", "12", "Arun", "900"]}'

Expected Output

Asset asset11 created successfully

Step 7: Read the Asset Details

Retrieve the information of asset11.

peer chaincode query \

-C mychannel \

-n basic \

-c '{"Args": ["ReadAsset", "asset11"]}'

Expected Output

```
{
  "ID": "asset11",
  "Color": "White",
  "Size": 12,
  "Owner": "Arun",
  "AppraisedValue": 900
}
```

Step 8: Update Asset Information

Modify the asset information.

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls \

--cafile \$ORDERER_CA \

-C mychannel \

-n basic \

```
-c '{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```

Expected Output

Asset Updated Successfully

Step 9: Transfer Asset Ownership

Transfer ownership of the asset to another user.

peer chaincode invoke \

```
-o localhost:7050 \
```

```
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls \
```

```
--cafile $ORDERER_CA \
```

```
-C mychannel \
```

```
-n basic \
```

```
-c '{"function":"TransferAsset","Args":["asset11","David"]}'
```

Expected Output

Ownership transferred successfully

Step 10: Verify the Updated Asset

Query the asset again to verify the updated owner.

peer chaincode query \

```
-C mychannel \
```

```
-n basic \
```

```
-c '{"Args":["ReadAsset","asset11"]}'
```

Expected Output

```
{  
  "ID":"asset11",  
  "Color":"Black",  
  "Size":15,  
  "Owner":"David",  
  "AppraisedValue":1200  
}
```

Step 11: Delete the Asset

Remove the asset from the ledger.

peer chaincode invoke \

```
-o localhost:7050 \
```

```
--ordererTLSHostnameOverride orderer.example.com \
```

```
--tls \
```

```
--cafile $ORDERER_CA \
```

```
-C mychannel \
```

```
-n basic \
```

```
-c '{"function":"DeleteAsset","Args":["asset11"]}'
```

Expected Output

Asset Deleted Successfully

Step 12: Verify Asset Deletion

Attempt to read the deleted asset.

peer chaincode query \

-C mychannel \

-n basic \

-c '{"Args":["ReadAsset","asset11"]}'

Expected Output

Error: asset11 does not exist

Step 13: Stop the Network

./network.sh down

Flow of Asset Management

Start Fabric Network

|



Deploy Asset Chaincode

|



Initialize Ledger

|



Query All Assets

|



Create New Asset

|



Read Asset

|



Update Asset

|



Transfer Ownership

|



Delete Asset

|



Verify Ledger

|



Stop Network

Sample Output

Fabric Network Started Successfully

Asset Chaincode Installed

Ledger Initialized

Assets Retrieved Successfully

New Asset Created

Asset Updated Successfully

Ownership Transferred Successfully

Asset Deleted Successfully

Ledger Verified Successfully

```
tharun@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
└─$ export PATH=$PATH:$HOME/fabric-dev/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-dev/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID=Org1MSP
export CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem
export CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

(tharun@kali)~[~/fabric-dev/fabric-samples/test-network]
└─$ export ORG2_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem

(tharun@kali)~[~/fabric-dev/fabric-samples/test-network]
└─$ export ORDERER_CA=$PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

(tharun@kali)~[~/fabric-dev/fabric-samples/test-network]
└─$ echo $ORDERER_CA
/home/tharun/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
└─$ echo $ORG2_TLS_ROOTCERT_FILE
/home/tharun/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem

(tharun@kali)~[~/fabric-dev/fabric-samples/test-network]
└─$ ./network.sh deployCC \
    -ccl javascript \
    -ccn basic \
    -ccp ../asset-transfer-basic/chaincode-javascript
chaincode committed successfully
You can verify the committed chaincode
using Docker:
peer lifecycle chaincode querycommitted -c mychannel -n basic

You should see something similar to:
Name: basic
Version: 1.0
Sequence: 1

Step 6 — Initialize the Ledger
└─$ ./network.sh init
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
```

```

Session Actions Edit View Help
Chaincode initialization is not required

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer lifecycle chaincode querycommitted -c mychannel -n basic
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTlsHostOverride orderer.example.com \
  --tls \
  --cafile "$ORDERER_CA" \
  -c mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  -c '{"function":"InitLedger","Args":[]}' \
  --waitForEvent
2026-08-28 18:07:39.393 IST 0001 INFO [chaincodeCmd] ClientWait → txid [30802d2afd82032e832c839ecb680441038d6a0a4e1315a723e478f576ec6810] committed with status (VALID) at localhost:9051
2026-08-28 18:07:39.393 IST 0002 INFO [chaincodeCmd] ClientWait → txid [30802d2afd82032e832c839ecb680441038d6a0a4e1315a723e478f576ec6810] committed with status (VALID) at localhost:7051
2026-08-28 18:07:39.393 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
  -c mychannel \
  -n basic \
  -c '{"function":"GetAllAssets","Args":[]}'
[{"AppraisedValue":200,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":600,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Mux","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTlsHostOverride orderer.example.com \
  --tls \
  --cafile "$ORDERER_CA" \
  -c mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  -c '{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}' \
  --waitForEvent
2026-08-28 18:08:13.279 IST 0001 INFO [chaincodeCmd] ClientWait → txid [9afdcad9f45dab208812302e0180d0f692c7839f52f42cc38034295df4187642] committed with status (VALID) at localhost:7051
2026-08-28 18:08:13.280 IST 0002 INFO [chaincodeCmd] ClientWait → txid [9afdcad9f45dab208812302e0180d0f692c7839f52f42cc38034295df4187642] committed with status (VALID) at localhost:9051
2026-08-28 18:08:13.280 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload:{"\type":"Buffer","\data":{"asset11":{"Color":"white","Size":12,"Owner":"Arun","AppraisedValue":900}}}

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
  -c mychannel \
  -n basic \
  -c '{"function":"ReadAsset","Args":["asset11"]}'
{"AppraisedValue":1200,"Color":"Black","ID":"asset11","Owner":"David","Size":15}

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
  -o localhost:7050 \
  --ordererTlsHostOverride orderer.example.com \
  --tls \
  --cafile "$ORDERER_CA" \
  -c mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
  -c '{"function":"DeleteAsset","Args":["asset11"]}' \
  --waitForEvent
2026-08-28 18:10:09.515 IST 0001 INFO [chaincodeCmd] ClientWait → txid [5b6dc7aaf151c1272a882a6a45cd6b4733a1cf8d7c63dff5077d976215ac024] committed with status (VALID) at localhost:7051
2026-08-28 18:10:09.516 IST 0002 INFO [chaincodeCmd] ClientWait → txid [5b6dc7aaf151c1272a882a6a45cd6b4733a1cf8d7c63dff5077d976215ac024] committed with status (VALID) at localhost:9051
2026-08-28 18:10:09.517 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200 payload:{"\type":"Buffer","\data":{"}}

(tharun@kali):~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
  -c mychannel \
  -n basic \
  -c '{"function":"ReadAsset","Args":["asset11"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset11 does not exist"

```

Result

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.